



UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA AUTOMATIKU I ELEKTRONIKU

Laboratorijska vježba 5

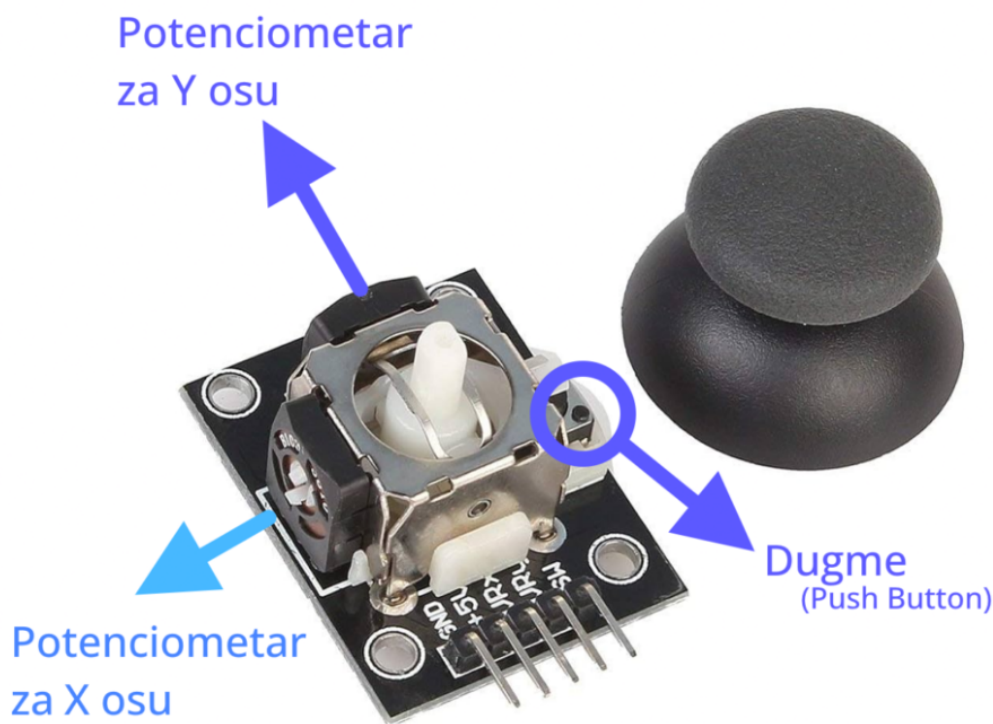
Praktikum iz logičkog dizajna i upravljanja zasnovan na FPGA razvojnom sistemu Gowin 9k

Tarik Ćatić

U ovoj vježbi proširuje se interaktivna HDMI igra iz vježbe 3 tako da se crvenom lopticom upravlja analognim džojstikom, čije se analogne vrijednosti očitavaju ADS1115 konvertorom putem I2C komunikacije.

Teorijski uvod – analogni džojstik i proširenje interaktivne HDMI igre

U laboratorijskoj vježbi 4 potencijometar je korišten kao izvor analognog napona koji se, preko ADS1115 modula i I2C magistrale, digitalizuje i koristi za upravljanje LED diodama. U ovoj vježbi isti princip očitavanja analogne veličine primjenjuje se na interaktivnu HDMI igru: umjesto tastera, položajem crvene loptice na ekranu upravlja se analognim džojstikom (slika 1), čime FPGA sistem prvi put istovremeno kombinuje generisanje video signala u realnom vremenu i obradu analognog upravljačkog signala.



Slika 1: Analogni džojstik modul korišten u laboratorijskoj vježbi.

Analogni džojstik kao dvoosni potenciometar

Analogni džojstik modul u suštini predstavlja dva nezavisna linearna potenciometra – jedan za X, drugi za Y osu – čiji su klizači mehanički spojeni na zajedničku polugu, tako da naginjanje poluge u bilo kojem pravcu istovremeno mijenja položaj oba klizača, srazmjerno projekciji otklona na svaku od osa [1]. Svaka osa se, sa stanovišta električnog kola, ponaša identično potenciometru opisanom u laboratorijskoj vježbi 4 – napon na klizaču dat je istim izrazom naponskog dijelila kao u LV4 – s tom razlikom da se u mirovanju (kada korisnik ne dodiruje polugu) klizač mehanički vraća u sredinu otporničke staze, pa napon na izlazu svake ose u mirovanju iznosi približno polovinu napona napajanja V_{DD} [1]. U ovoj vježbi korištena je samo jedna osa džojstika (Y), čiji se izlaz dovodi na isti analogni ulaz ADS1115 modula (A0) koji je u LV4 korišten za potenciometar. Druga osa (X) ostaje nespojena, a horizontalna pozicija loptice je fiksna.

Generisanje HDMI video signala i logika igre

Prikaz na monitoru ostvaruje se generisanjem HDMI video signala rezolucije 1280×720 piksela pri frekvenciji osvježavanja od 60 Hz, uz TMDS (engl.: *Transition-Minimized Differential Signaling*) kodiranje i serijalizaciju piksela u FPGA-u – ova infrastruktura (hdmi_backend sa pripadajućim podmodulima) preuzeta je nepromijenjena iz ranije razvijenog projekta i u ovoj vježbi se koristi kao gotova cjelina, bez izmjena. Iznad generisanog video signala implementirana je jednostavna igra: crvena loptica kružnog oblika kreće se unutar igraćeg polja, dok se zeleni vertikalni zid sa horizontalnim prorezom kontinuirano pomjera zdesna nalijevo, sa novom (slučajno odabranom) visinom proreza pri svakom novom pojavljivanju sa desne strane ekrana. Zadatak igrača je da džojstikom pomjera lopticu gore/dole tako da ona u trenutku susreta sa zidom prođe kroz otvoreni prorez; u suprotnom dolazi do sudara, što se signalizira

posebnom LED diodom. Fokus ove vježbe nije na samom generisanju HDMI signala (koje je pretpostavljeno kao već savladano), već na proširenju postojeće igre novim, analognim načinom upravljanja.

Mrtva zona i prelazak između taktnih domena

Za razliku od tastera, koji imaju samo dva stanja (pritisnut/otpušten), analogni džojstik nikada ne miruje u savršeno istom položaju – mehaničke tolerancije i sitan šum uzrokuju da očitana vrijednost oko sredine oscilira za nekoliko desetina do stotina jedinica ADC koda, čak i kada korisnik uopšte ne dodiruje polugu. Kada bi se i najmanji otklon od teorijske sredine odmah pretvarao u pomjeranje loptice, ona bi neprestano, neželjeno "puzala" u jednom pravcu. Ovaj problem se rješava uvođenjem tzv. mrtve zone (engl.: *deadzone*) – opsega vrijednosti oko centra unutar kojeg se očitavanje tumači kao "džojstik u mirovanju", opisano detaljnije u sekciji 1.

Dodatna razlika u odnosu na LV4 jeste da se u ovoj vježbi po prvi put unutar jednog projekta koriste dva nezavisna taktna domena: I2C dio (kao i cijeli LV4) radi na frekvenciji vanjskog oscilatora od 27 MHz, dok HDMI dio, radi generisanja piksela odgovarajućom brzinom, koristi izvedeni takt od 74.25 MHz, dobijen internim PLL-om. Signal koji nastaje u jednom taktnom domenu (rezultat očitavanja džojstika) ne smije se direktno koristiti kao ulaz u logiku drugog taktnog domena (pomjeranje loptice) – takav prelazak, poznat kao prelazak između taktnih domena (engl.: *Clock Domain Crossing*, CDC), bez odgovarajuće sinhronizacije može dovesti do metastabilnosti: stanja u kojem izlaz flip-flopa privremeno poprimi neodređenu, ni logičku nulu ni jedinicu vrijednost, usljed kršenja vremena postavljanja/držanja (engl.: *setup/hold time*) signala koji se mijenja asinhrono u odnosu na takt koji ga uzorkuje [2]. Standardno i u praksi najčešće korišteno rješenje za spore, jednobitne (ili, kao ovdje, uskopojasne) signale jeste dvostruki flip-flop sinhronizator (engl.: *double flip-flop synchronizer*): signal se registruje sa dva uzastopna flip-flopa u odredišnom taktnom domenu, čime se vjerovatnoća da metastabilnost "procuri" dalje kroz kolo svodi na zanemarljivo malu vrijednost, po cijenu kašnjenja od jednog dodatnog perioda odredišnog takta [2]. Primjena ovog rješenja u konkretnoj implementaciji opisana je u sekciji 1.

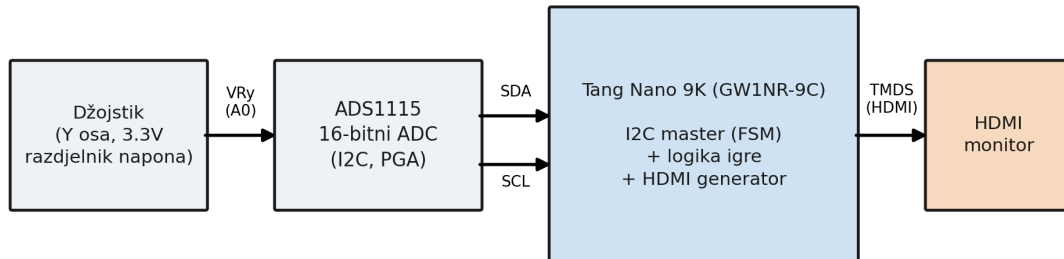
Struktura projekta i način spajanja

Projekat korišten u ovoj vježbi sastoji se od sljedećih fajlova:

- `i2c_ctrl sv` – modul I2C *master*-a, potpuno nepromijenjen u odnosu na LV4: isti FSM, ista konfiguracija ADS1115-a (jednostruko mjerenje na A0, $PGA = \pm 2.048 V$).
- `hdmi_pkg sv`, `hdmi_backend sv`, `hdmi_tdms_enc sv`, `fpgatech_lib/GOWIN/* sv` – infrastruktura za generisanje HDMI video signala (PLL, serijalizatori, TMDS enkoder), preuzeta nepromijenjena iz ranijeg projekta.
- `top sv` – najviši nivo hijerarhije: instancira HDMI generator i I2C *master*, implementira logiku igre (loptica, prepreka, detekcija sudara) i, novo u odnosu na raniju verziju igre, preslikava očitavanje džojstika u pomjeranje loptice po Y osi.
- `projekat cst` – fizičko mapiranje signala na pinove Tang Nano 9K razvojnog sistema.

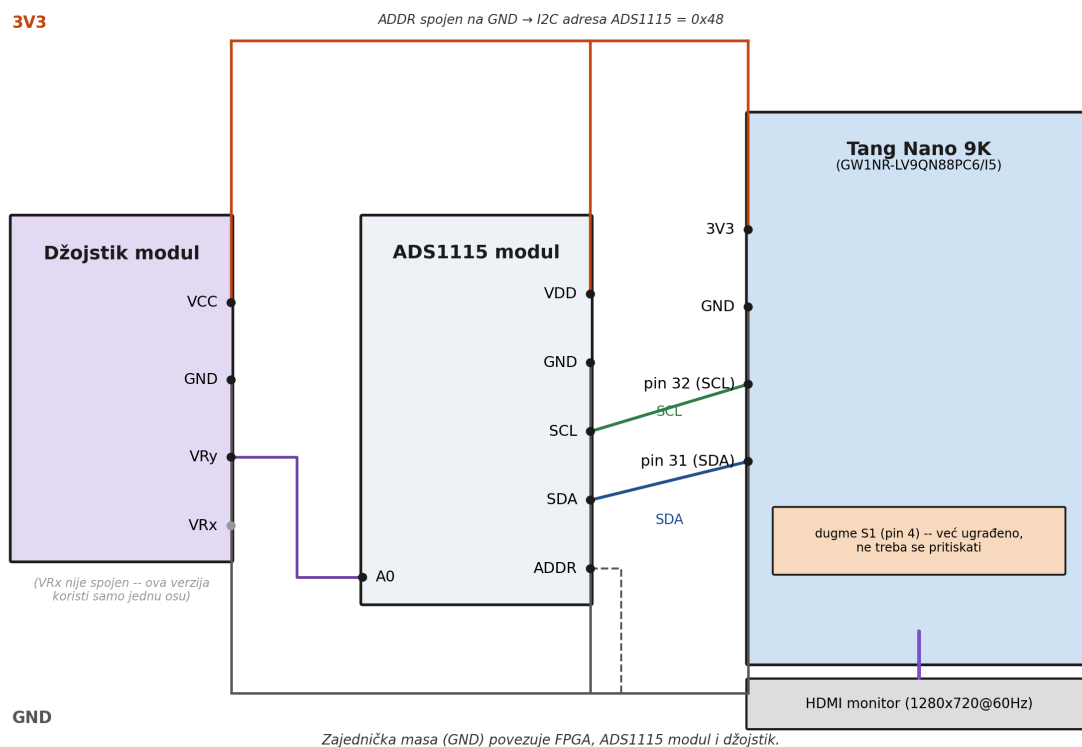
Blok dijagram sistema prikazan je na slici 2: džojstik (samo Y osa) spojen je na ADS1115 modul, koji FPGA sistem naizmjenično čita I2C protokolom; FPGA istovremeno generiše HDMI video signal koji nosi sliku igre ka monitoru.

Blok dijagram sistema: Y osa džojstika se čita preko I2C busa i koristi za pomjeranje crvene loptice u igri prikazanoj putem HDMI-a



Slika 2: Blok dijagram sistema korištenog u laboratorijskoj vježbi 5.

Fizičko spajanje džojstika i ADS1115 modula na Tang Nano 9K razvojni sistem prikazano je na slici 3. Spoj je identičan onom iz LV4 (ista I2C konfiguracija, isti pinovi 31/32, potencijometar zamijenjen Y osom džojstika na istom ulazu A0), uz jednu dodatnu, važnu sličnost: signal reseta I2C/džojstik dijela kola (`areset_n`) vezan je, kao i u LV4, na fizičko dugme S1 (pin 4) razvojne pločice – dio kola koji radi na 27 MHz tako ima svoj sopstveni, nezavisan izvor reseta, potpuno odvojen od reseta HDMI dijela kola, čime se izbjegava potreba za dodatnim sinhronizatorom reset signala između dva taktna domena.



Slika 3: Šema spajanja džojstika, ADS1115 modula i Tang Nano 9K razvojnog sistema.

Fizičko mapiranje pinova dato je u Kodu 1. HDMI izlazni pinovi (diferencijalni parovi takta i tri kanala podataka) preuzeti su nepromijenjeni iz ranijeg projekta.

Kod 1 Fizičko mapiranje pinova za laboratorijsku vježbu 5 (projekat.cst).

```
IO_LOC   "clk_ext" 52;
IO_PORT  "clk_ext" PULL_MODE=UP   IO_TYPE=LVC MOS33 ;

IO_LOC   "areset_n" 4;
IO_PORT  "areset_n" IO_TYPE=LVC MOS18 PULL_MODE=UP;

IO_LOC   "i2c_sda" 31;
IO_PORT  "i2c_sda" IO_TYPE=LVC MOS33;
IO_LOC   "i2c_scl" 32;
IO_PORT  "i2c_scl" IO_TYPE=LVC MOS33;

IO_LOC   "led_n" 11;
IO_PORT  "led_n" PULL_MODE=UP DRIVE=8;

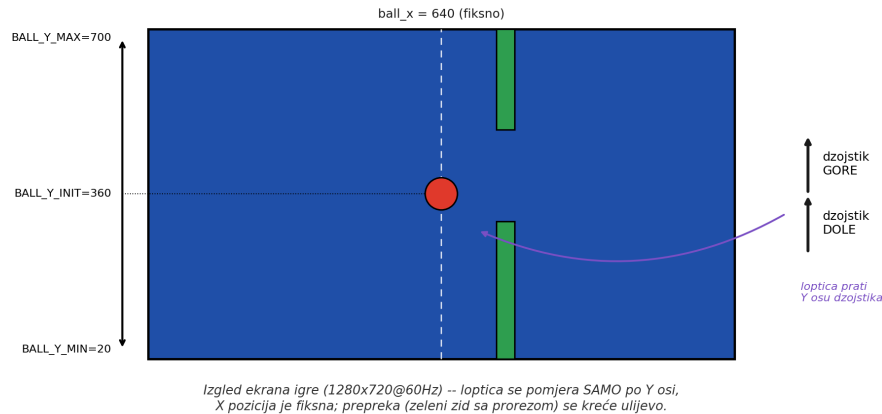
IO_LOC   "led_hit_n" 10;
IO_PORT  "led_hit_n" PULL_MODE=UP DRIVE=8;

IO_LOC   "led_dbg_n[0]" 13;
IO_PORT  "led_dbg_n[0]" PULL_MODE=UP DRIVE=8;
IO_LOC   "led_dbg_n[1]" 14;
IO_PORT  "led_dbg_n[1]" PULL_MODE=UP DRIVE=8;
IO_LOC   "led_dbg_n[2]" 15;
IO_PORT  "led_dbg_n[2]" PULL_MODE=UP DRIVE=8;

IO_LOC   "hdmi_clk_p"      69,68;
IO_PORT  "hdmi_clk_p"      PULL_MODE=NONE DRIVE=8;
IO_LOC   "hdmi_dat_p[0]" 71,70;
IO_PORT  "hdmi_dat_p[0]" PULL_MODE=NONE DRIVE=8;
IO_LOC   "hdmi_dat_p[1]" 73,72;
IO_PORT  "hdmi_dat_p[1]" PULL_MODE=NONE DRIVE=8;
IO_LOC   "hdmi_dat_p[2]" 75,74;
IO_PORT  "hdmi_dat_p[2]" PULL_MODE=NONE DRIVE=8;
```

Zadatak – upravljanje igrom putem Y ose analognog džojstika

Zadatak je proširiti postojeću igru (crvena loptica, pokretna prepreka sa prorezom, detekcija sudara) tako da se loptica po vertikalnoj (Y) osi pomjera analognim džojstikom umjesto tastetima, dok njena horizontalna (X) pozicija ostaje fiksna, na sredini ekrana. Izgled ekrana igre, sa označenim granicama kretanja loptice, prikazan je na slici 4.



Slika 4: Izgled ekrana igre – loptica se kreće isključivo po Y osi, između granica BALL_Y_MIN i BALL_Y_MAX.

Implementacija zadatka data je u Kodu 2.

Kod 2 SystemVerilog implementacija upravljanja igrom putem Y ose džojstika (top . sv).

```

1 module top
2   import hdmi_pkg::*;
3   (
4     input  logic  clk_ext,      // vanjski oscilator: 27 MHz (Tang Nano
                                9K)
5     input  logic  areset_n,     // fizicko dugme S1 (aktivno nisko) ->
                                pin 4
6
7     inout  wire   i2c_scl,      // ka ADS1115 SCL
8     inout  wire   i2c_sda,      // ka ADS1115 SDA
9
10    output logic  hdmi_clk_p,
11    output logic  hdmi_clk_n,
12    output bus3_t hdmi_dat_p,
13    output bus3_t hdmi_dat_n,
14
15    output logic  led_n,         // treperi ravnomjerno - igra je "ziva"
16    output logic  led_hit_n,     // pali se isključivo pri sudaru sa
                                preprekom
17
18    output logic [2:0] led_dbg_n // dijagnostički bar-graf (vidi
                                tekst)
19  );
20
21  logic  clk_pix, srst_n;
22  logic  frame_done;
23  logic  mfram_done;
24
25  bus12_t x, y;
26  pix_t  pix;

```

```

27 // Loptica: X pozicija fiksna, Y pozicija se pomjera dzojstikom.
28 localparam bus12_t BALL_R      = 12'd16;
29 localparam bus12_t BALL_X_INIT = 12'd640;
30 localparam bus12_t BALL_Y_MIN  = 12'd20;
31 localparam bus12_t BALL_Y_MAX  = 12'd700;
32 localparam bus12_t BALL_Y_INIT = 12'd360;
33
34
35 bus12_t ball_y;
36 localparam bus12_t ball_x = BALL_X_INIT;
37
38 // Prepreka: vertikalni zid sirine OBST_WIDTH, sa prorezom
39 //          GAP_HEIGHT,
40 // koji se krece OBST_SPEED piksela po prikazanom okviru.
41 localparam bus12_t OBST_WIDTH  = 12'd40;
42 localparam bus12_t OBST_SPEED  = 12'd6;
43 localparam bus12_t GAP_HEIGHT  = 12'd200;
44
45 bus12_t obst_x;
46 bus12_t gap_y;
47 logic [1:0] gap_sel;
48
49 function automatic bus12_t gap_preset(input logic [1:0] sel);
50     case (sel)
51         2'd0: gap_preset = 12'd60;
52         2'd1: gap_preset = 12'd220;
53         2'd2: gap_preset = 12'd380;
54         default: gap_preset = 12'd500;
55     endcase
56 endfunction
57
58 logic obst_here, in_gap, hit;
59
60 always_comb begin: _my_game
61     obst_here = (x >= obst_x) && (x < obst_x + OBST_WIDTH);
62     in_gap     = (y >= gap_y) && (y < gap_y + GAP_HEIGHT);
63
64     if ((x-ball_x)*(x-ball_x)+(y-ball_y)*(y-ball_y)<=250) begin
65         pix.R = '1; pix.G = '0; pix.B = '0; // loptica
66     end else if (obst_here && !in_gap) begin
67         pix.R = '0; pix.G = '1; pix.B = '0; // prepreka
68     end else begin
69         pix.R = '0; pix.G = '0; pix.B = '1; // pozadina
70     end
71 end: _my_game
72
73 always_comb begin: _collision_check
74     hit = (obst_x <= ball_x + BALL_R)
75         & (obst_x + OBST_WIDTH > ball_x - BALL_R)
76         & ~( (ball_y >= gap_y + BALL_R) & (ball_y + BALL_R <= gap_y
77             + GAP_HEIGHT) );

```

```

76  end: _collision_check
77
78  hdmi_backend u_hdmi_backend (
79      .clk_ext(clk_ext), .clk_pix(clk_pix), .srst_n(srst_n),
80      .hcount(x), .vcount(y), .pix(pix),
81      .frame_done(frame_done), .mfram_done(mfram_done),
82      .hdmi_clk_p(hdmi_clk_p), .hdmi_clk_n(hdmi_clk_n),
83      .hdmi_dat_p(hdmi_dat_p), .hdmi_dat_n(hdmi_dat_n)
84  );
85
86  always_ff @(negedge srst_n or posedge clk_pix) begin:
    _obstacle_mov
87      if (srst_n == L0) begin
88          obst_x  <= 12'(HSCREEN) + OBST_WIDTH;
89          gap_y   <= gap_preset(2'd0);
90          gap_sel <= 2'd0;
91      end else if (frame_done == HI) begin
92          if (obst_x <= OBST_SPEED) begin
93              gap_sel <= gap_sel + 2'd1;
94              gap_y   <= gap_preset(gap_sel + 2'd1);
95              obst_x  <= 12'(HSCREEN) + OBST_WIDTH;
96          end else begin
97              obst_x <= obst_x - OBST_SPEED;
98          end
99      end
100  end: _obstacle_mov
101
102  always_ff @(negedge srst_n or posedge clk_pix) begin: _led_mon
103      if (srst_n == L0) led_n <= HI;
104      else if (mfram_done == HI) led_n <= ~led_n;
105  end: _led_mon
106
107  always_ff @(negedge srst_n or posedge clk_pix) begin: _hit_led
108      if (srst_n == L0) led_hit_n <= HI;
109      else led_hit_n <= hit ? L0 : HI;
110  end: _hit_led
111
112  // -----
113  // I2C / ADS1115 / dzojstik -- identicno LV4 (jedan kanal, AIN0,
114  // PGA=+-2.048V), radi u domenu takta clk_ext (27 MHz).
115  // -----
116  logic [26:0] brojac_i2c;
117  logic strobe_400kHz;
118
119  always_ff @(negedge areset_n or posedge clk_ext) begin
120      if (areset_n == 1'b0) begin
121          brojac_i2c    <= 27'd0;
122          strobe_400kHz <= 1'b0;
123      end else begin
124          brojac_i2c <= brojac_i2c + 27'd1;
125          if (brojac_i2c == 27'd68) begin

```

```

126         strobe_400kHz <= 1'b1;
127         brojac_i2c     <= 27'd0;
128     end else begin
129         strobe_400kHz <= 1'b0;
130     end
131 end
132 end
133
134 logic transaction_done;
135 logic [15:0] result_16;
136 logic scl_oe, scl_di, sda_oe, sda_di;
137
138 i2c_ctrl u_i2c_ads1115 (
139     .clk(clk_ext), .strobe_400kHz(strobe_400kHz), .areset_n(
140         areset_n),
141     .start_transaction(1'b1), .pause_duration(16'hFFFF),
142     .transaction_done(transaction_done),
143     .scl_oe(scl_oe), .scl_di(scl_di), .sda_oe(sda_oe), .sda_di(
144         sda_di),
145     .result_16(result_16)
146 );
147
148 assign i2c_scl = scl_oe ? 1'b0 : 1'bz;
149 assign scl_di  = i2c_scl;
150 assign i2c_sda = sda_oe ? 1'b0 : 1'bz;
151 assign sda_di  = i2c_sda;
152
153 logic [15:0] adc_mag;
154 assign adc_mag = result_16[15] ? 16'd0 : result_16;
155
156 // Dijagnosticki bar-graf: [0],[1] -- pragovi sirove magnitude;
157 // [2] -- "otkucaj srca" FSM-a (mijenja stanje pri svakoj
158 //      završenoj
159 // I2C transakciji), koristan za brzu provjeru da protokol uopšte
160 //      radi.
161 logic [2:0] led_dbg;
162 assign led_dbg[0] = (adc_mag >= 16'd10000);
163 assign led_dbg[1] = (adc_mag >= 16'd22000);
164
165 logic i2c_heartbeat;
166 always_ff @(negedge areset_n or posedge clk_ext) begin
167     if (areset_n == 1'b0) i2c_heartbeat <= 1'b0;
168     else if (transaction_done) i2c_heartbeat <= ~i2c_heartbeat;
169 end
170 assign led_dbg[2] = i2c_heartbeat;
171 assign led_dbg_n  = ~led_dbg;
172
173 // Preslikavanje ADC koda u brzinu/pravac pomjeranja loptice.
174 localparam logic signed [16:0] JOY_CENTER  = 17'sd26400;
175 localparam logic          [15:0] DEADZONE   = 16'd800;
176 localparam logic          [15:0] FAST_THRESH = 16'd4000;

```

```

173 localparam logic signed [ 4:0] STEP_SLOW    = 5'sd3;
174 localparam logic signed [ 4:0] STEP_FAST    = 5'sd9;
175
176 logic signed [16:0] diff_y;
177 assign diff_y = $signed({1'b0, adc_mag}) - JOY_CENTER;
178
179 logic [15:0] adiff_y;
180 assign adiff_y = diff_y[16] ? 16'(-diff_y) : 16'(diff_y);
181
182 logic signed [4:0] step_y_comb;
183 always_comb begin
184     if (adiff_y < DEADZONE) begin
185         step_y_comb = 5'sd0;
186     end else if (adiff_y < FAST_THRESH) begin
187         step_y_comb = diff_y[16] ? -STEP_SLOW : STEP_SLOW;
188     end else begin
189         step_y_comb = diff_y[16] ? -STEP_FAST : STEP_FAST;
190     end
191 end
192
193 logic signed [4:0] step_y_ext;
194 always_ff @(negedge areset_n or posedge clk_ext) begin
195     if (areset_n == 1'b0) step_y_ext <= 5'sd0;
196     else step_y_ext <= step_y_comb;
197 end
198
199 // CDC: dvostruki flip-flop sinhronizator iz clk_ext u clk_pix
200 // domen.
201 logic signed [4:0] step_y_sync0, step_y_sync;
202 always_ff @(negedge srst_n or posedge clk_pix) begin:
203     _cdc_sync_pix
204     if (srst_n == L0) begin
205         step_y_sync0 <= 5'sd0;
206         step_y_sync <= 5'sd0;
207     end else begin
208         step_y_sync0 <= step_y_ext;
209         step_y_sync <= step_y_sync0;
210     end
211 end: _cdc_sync_pix
212
213 always_ff @(negedge srst_n or posedge clk_pix) begin: _ball_mov
214     if (srst_n == L0) begin
215         ball_y <= BALL_Y_INIT;
216     end else if (frame_done == HI) begin
217         if (step_y_sync > 5'sd0) begin
218             if (ball_y < BALL_Y_MAX - 12'(step_y_sync))
219                 ball_y <= ball_y + 12'(step_y_sync);
220             else
221                 ball_y <= BALL_Y_MAX;
222         end else if (step_y_sync < 5'sd0) begin
223             if (ball_y > BALL_Y_MIN + 12'(-step_y_sync))

```

```

222         ball_y <= ball_y - 12' (-step_y_sync);
223     else
224         ball_y <= BALL_Y_MIN;
225     end
226 end
227 end: _ball_mov
228
229 endmodule: top

```

U nastavku je detaljnije opisana uloga pojedinih dijelova koda datog u Kodu 2.

Portovi i preuzeta HDMI/logika igre

Ulazno-izlazni portovi modula `top` prošireni su u odnosu na raniju verziju igre (koja je koristila dva tastera) sa signalima `areset_n`, `i2c_scl` i `i2c_sda`, dok su `hdmi_clk_p/n`, `hdmi_dat_p/n`, `led_n` i `led_hit_n` nepromijenjeni. Instanciranje `hdmi_backend` (koji generiše taktove `clk_pix` i `srst_n`, te trenutnu poziciju piksela `x/y`), blok `_my_game` (koji boji trenutni piksel u zavisnosti od toga da li se nalazi unutar loptice, prepreke ili pozadine), blok `_collision_check` (koji izračunava signal `hit`), blok `_obstacle_mov` (koji svakih 60-tak puta u sekundi pomjera prepreku ulijevo) i blokovi `_led_mon/_hit_led` (LED signalizacija) preuzeti su bez izmjena iz ranije razvijene igre – jedina razlika je zamjena fiksne konstante `BALL_X` promjenljivom `ball_y`, koja se, umjesto tasterima, sada ažurira iz I2C/džojstik dijela koda opisanog u nastavku.

Nezavisan reset I2C/džojstik dijela kola

Dio koda koji čita džojstik radi u domenu takta `clk_ext` (27 MHz) i, kao i u LV4, koristi `areset_n` – signal sa fizičkog dugmeta S1 – kao svoj asinhroni reset. Ovaj signal je namjerno potpuno nezavisan od signala `srst_n` koji resetuje HDMI dio kola (domen takta `clk_pix`): s obzirom na to da su u pitanju dva odvojena taktna domena, deljenje jednog te istog reset signala između njih bi samo po sebi predstavljalo još jedan slučaj prelaska između taktnih domena (jer bi se oslobađanje reseta u jednom domenu moralo bezbjedno prenijeti i u drugi domen), pa je jednostavnije i pouzdanije svakom taktnom domenu dodijeliti sopstveni, nezavisan izvor reseta.

Generisanje I2C takta i čitanje ADS1115 (identično LV4)

Blok koji generiše `strobe_400kHz` i instanciranje modula `i2c_ctrl` (`u_i2c_ads1115`) potpuno su identični odgovarajućem dijelu koda iz LV4, uključujući i vrijednost `pause_duration=16'hFF` – detaljno objašnjenje ovog dijela dato je u zadatku 1 laboratorijske vježbe 4. Isto tako, tri-state upravljanje linijama `i2c_scl/i2c_sda` i odsijecanje eventualnih negativnih kodova u signalu `adc_mag` preuzeti su nepromijenjeni iz LV4.

Dijagnostički bar-graf

Tri diode `led_dbg_n[2:0]` nisu neophodne za rad same igre, ali predstavljaju koristan alat za nezavisnu provjeru rada I2C dijela kola tokom razvoja i testiranja, potpuno odvojeno od HDMI slike: diode `led_dbg_n[0]` i `led_dbg_n[1]` pale se kada sirova magnituda očitavanja `adc_mag` pređe unaprijed određene pragove, dok `led_dbg_n[2]` mijenja stanje pri

svakom signalu `transaction_done` – s obzirom na to da se taj signal generiše dva puta u toku svakog potpunog ciklusa čitanja (jednom nakon upisa konfiguracije, jednom po završetku čitanja rezultata), ova dioda vidljivo treperi sve dok I2C komunikacija ispravno funkcioniše, a ostaje trajno u jednom stanju ukoliko FSM iz bilo kojeg razloga (npr. izostanak ACK signala od ADS1115 modula) ne uspije završiti transakciju. Ovakav pristup – ugrađivanje jednostavnih, nezavisnih dijagnostičkih signala u sâm dizajn – česta je i korisna praksa pri otklanjanju grešaka u digitalnim sistemima, jer omogućava lociranje problema (u ovom slučaju: da li greška potiče iz same I2C komunikacije ili iz logike koja obrađuje već ispravno očitane vrijednosti) bez potrebe za spoljašnjom mjernom opremom.

Mrtva zona i preslikavanje u brzinu pomjeranja

Rezultat `adc_mag` se prvo upoređuje sa teorijskom vrijednošću sredine, `JOY_CENTER`. Budući da je odabrani PGA opseg $FS = \pm 2.048\text{ V}$ (isti kao u LV4), a džojstik se, iz istog sigurnosnog razloga kao potencijometar u LV4, napaja sa 3.3 V FPGA sabirnice, teorijski napon sredine iznosi $V_{sredina} = 3.3/2 = 1.65\text{ V}$, što prema izrazu za pretvorbu ADS1115 modula iz LV4 odgovara digitalnom kodu:

$$JOY_CENTER = \frac{V_{sredina}}{FS} \cdot 2^{15} = \frac{1.65}{2.048} \cdot 32768 \approx 26400 \quad (1)$$

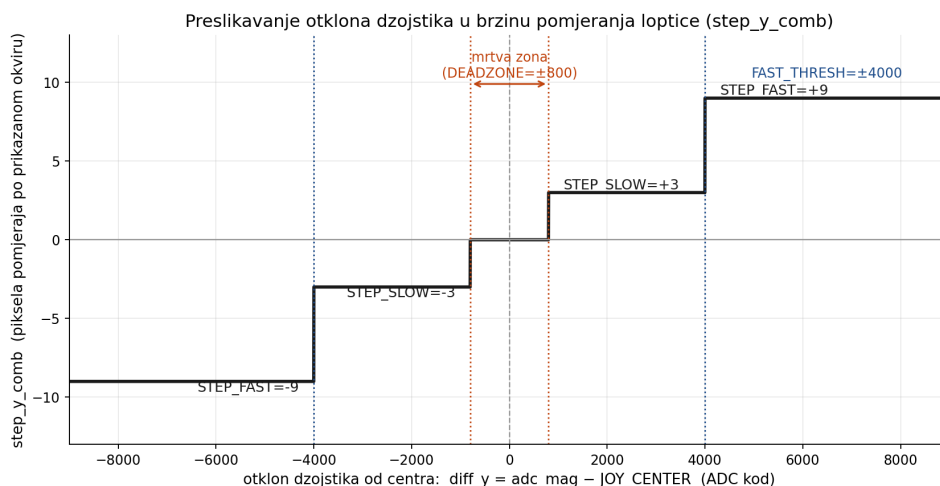
Razlika `diff_y = adc_mag - JOY_CENTER` (računata u dvostrukom komplementu, sa dodatnim bitom radi ispravnog predznaka) predstavlja potpisani otklon trenutnog očitavanja od te teorijske sredine, dok `adiff_y` predstavlja njegovu apsolutnu vrijednost. Na osnovu `adiff_y` i predznaka `diff_y`, brzina i smjer pomjeranja loptice po Y osi (`step_y_comb`) određuju se prema pravilu:

$$\text{step_y_comb} = \begin{cases} 0, & |\text{diff_y}| < DEADZONE \\ \pm STEP_SLOW, & DEADZONE \leq |\text{diff_y}| < FAST_THRESH \\ \pm STEP_FAST, & |\text{diff_y}| \geq FAST_THRESH \end{cases} \quad (2)$$

gdje predznak $+$ odgovara otklonu džojstika iznad sredine (`diff_y[16]=0`), a predznak $-$ otklonu ispod sredine (`diff_y[16]=1`). Konkretno vrijednosti parametara korištene u implementaciji date su u tabeli 1, a sama funkcija je grafički prikazana na slici 5.

Tabela 1: Parametri preslikavanja otklona džojstika u brzinu pomjeranja loptice.

Parametar	Vrijednost	Značenje
JOY_CENTER	26400	teorijska sredina (jednačina (1))
DEADZONE	800	poluširina mrtve zone oko centra
FAST_THRESH	4000	prag otklona za prelazak na veću brzinu
STEP_SLOW	3 piksela/okvir	brzina za mali otklon (mrtva zona $\leq diff < FAST_THRESH$)
STEP_FAST	9 piksela/okvir	brzina za veliki otklon ($ diff \geq FAST_THRESH$)



Slika 5: Preslikavanje otklona dżojstika od centra u brzinu pomjeranja loptice, sa označenom mrtvom zonom i pragom prelaska na veću brzinu.

Vrijednost `step_y_comb` se registruje u signal `step_y_ext` u istom taktom domenu (`clk_ext`) prije prelaska u domen `clk_pix` – ovo je standardna CDC praksa: sinhronizuje se već registrovan, a ne "sirov" kombinacioni signal, čime se izbjegavaju dodatni hazardi (kratkotrajne, nevažeće međuvrijednosti kombinacione logike) na ulazu sinhronizatora [2].

Prelazak u domen `clk_pix` i pomjeranje loptice

Blok `_cdc_sync_pix` implementira dvostruki flip-flop sinhronizator opisan u sekciji : signal `step_y_ext`, nastao u domenu `clk_ext`, registruje se dva puta uzastopno na takt `clk_pix` (`step_y_sync0`, zatim `step_y_sync`), čime se rizik od metastabilnosti svodi na zanemarljiv nivo prije nego što se signal upotrijebi u logici koja pomjera lopticu [2]. Konačno, blok `_ball_mov` – koji se, kao i pomjeranje prepreke, izvršava jednom po svakom prikazanom okviru (signal `frame_done`, 60 puta u sekundi) – ažurira `ball_y` za iznos `step_y_sync` u odgovarajućem smjeru, uz provjeru granica `BALL_Y_MIN/BALL_Y_MAX`: ukoliko bi pomjeranje za puni korak izašlo van dozvoljenog opsega, loptica se umjesto toga postavlja tačno na najbližu granicu, čime se sprječava da izađe van vidljivog dijela ekrana.

Testiranje

Implementacija je testirana direktno na fizičkom sistemu, spojenom prema šemi na slici 3. Pomjeranjem dżojstika gore/dole potvrđeno je da se loptica pomjera u očekivanom smjeru i očekivanom (proporcionalnom otklonu) brzinom, da ostaje na granicama igraceg polja kada se dżojstik drži u krajnjem položaju, te da se, kada je otpušten, loptica zaustavlja i ostaje nepomična (zahvaljujući mrtvoj zoni iz jednačine (2)). Dijagnostička dioda `led_dbg_n[2]` vidljivo treperi tokom rada, potvrđujući da I2C komunikacija kontinuirano i uspješno funkcioniše, dok diode `led_dbg_n[1:0]` prate otklon dżojstika od centra. Sudar loptice sa preprekom ispravno aktivira diodu `led_hit_n`, dok dioda `led_n` tokom cijelog rada ravnomjerno treperi, potvrđujući da HDMI dio kola radi neprekidno i nezavisno od I2C/dżojstik dijela.

Literatura

- [1] Last Minute Engineers, "In-depth: How a 2-axis joystick works? interfacing with arduino & processing", lastminuteengineers.com, dostupno na: <https://lastminuteengineers.com/joystick-interfacing-arduino-processing/>
- [2] Cummings, C. E., "Clock domain crossing (cdc) design & verification techniques using SystemVerilog", Synopsys Users Group (SNUG), Boston, MA, Conference paper, 2008, dostupno na: http://www.sunburst-design.com/papers/CummingsSNUG2008Boston_CDC.pdf